



XML : Cours et exercices corrigés

Master Web Intelligence et Sciences des Données (WISD)

Prof. El Habib NFAOUI
(elhabib.nfaoui@usmba.ac.ma)

Edition 2020

Avant-propos

Ce polycopié est utilisé pour une partie du cours intitulé « Echange de données et Interopérabilité » du Master Web Intelligence et Sciences des Données (WISD). Ce polycopié se focalise particulièrement sur le rôle que joue le langage universel XML (eXtensible Markup Language) pour représenter les données échangées sur le Web. XML permet au développeur de délivrer du contenu depuis les applications à d'autres applications ou aux navigateurs. Ce polycopié se compose de quatre chapitres.

Le premier chapitre introduit le langage XML, la structure des documents XML ainsi que la notion d'espace de nom. Les deux chapitres qui suivent se focalisent sur la validation des documents XML. En particulier, le deuxième chapitre présente le langage DTD (Document Type Definition) qui permet de définir le « vocabulaire » et la structure qui seront utilisés dans un document XML. Il explique les bases de construction d'un DTD. Le chapitre 3 étudie les schémas XML (XSD) qui combleront les lacunes DTD. Il montre la syntaxe XSD et les bases de construction d'un schéma XML. Enfin, le chapitre 4 présente dix exercices avec leurs solutions.

Table des matières

Chapitre 1	5
Structure des documents XML	5
1. Introduction	5
2. Structure d'un document XML	5
3. Les espaces de noms	12
 Chapitre 2	 14
DTD (Document Type Definition).....	14
1. Introduction	14
2. Déclaration de type de document	14
3. Bases de la construction de DTD	15
3.1 Déclaration d'un élément	16
3.2 Déclaration des attributs d'un élément	19
 Chapitre 3	 23
XML Schema	23
1. Insuffisance des DTD	23
2. XML Schema : Définitions de base.....	23
3. Bases de la construction de XML Schema	23
3.1 Types de données XSD	24
3.2 Schémas XSD et documents XML	26
3.3 Types simples	26
3.4 Détails des types complexes	27
 Chapitre 4	 32
Exercices corrigés.....	32
1. Série d'exercices 1 : Structuration des données en XML	32
2. Série d'exercices 1 : Corrigé	35
3. Série d'exercices 2 : DTD (Document Type Definition)	42

4.	Série d'exercices 2 : Corrigé	43
5.	Série d'exercices 3 : XSD (XML Schema Language)	46
6.	Série d'exercices 3 : Corrigé	48

Chapitre 1

Structure des documents XML

1. Introduction

XML est un méta-langage universel pour représenter les données échangées sur le Web qui permet au développeur de délivrer du contenu depuis les applications à d'autres applications ou aux navigateurs. XML fournit simplement une structure fondamentale et un ensemble de règles qui doivent être respectées par tous les langages de balisage.

XML standardise la manière dont l'information est :

- échangée
- présentée
- archivée
- retrouvée
- transformée
- ...

2. Structure d'un document XML

Un document XML est un document texte lisible. Un **document** XML est découpé en **éléments** structurés hiérarchiquement. Un document a un élément racine appelé **élément du document**.

□ Un élément est composé :

- d'un **nom** qui spécifie son type,
- d'**attributs**,
- d'un **contenu** formé d'**éléments** ou de **textes**.

```
<producteur>  
  <adresse>  
    <rue>Azilal</rue>  
    <ville>Casa</ville>
```

```
</adresse>
</producteur>
```

Syntaxiquement, les éléments d'un document XML sont marqués dans le document lui-même par des paires de **balises** ouvrantes et fermantes.

Forme : <balise> de début, </balise> de fin

Exemple d'un document XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!-- Date de création : 12/10/09 -->
<cours titre="JEE">
    <intervenant nom="NFAOUI">
    </intervenant>
    <plan>
        Introduction
        JEE: Spécifications
    </plan>
</cours>
```

2.1 L'en-tête : le prologue

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

Il s'agit de la première ligne d'un document XML servant à donner les caractéristiques globales du document, c'est-à-dire :

- La version XML,
- Le jeu de caractères employé (*encoding*): encoding). Pour fonctionner, le parseur va avoir besoin de distinguer le rôle de chaque caractère, certains étant réservés à la syntaxe et d'autres représentant des données. Pour définir un jeu de caractères, XML s'appuie sur des standards ISO et Unicode (voir <http://www.unicode.org/>).
- Le champ **standalone** désigne l'indépendance du document, au sens où il n'existe aucun élément externe qui puisse altérer la forme finale du document XML fourni à l'application par le parseur (références d'entités, valeurs par défaut...). Ce champ prend les valeurs yes ou no.

2.2 Les instructions de traitement, les commentaires

Les instructions de traitement (*processing instruction* ou PI) n'ont pas de rôle lié aux données ou à la structuration de votre document. Elles servent à donner à l'application qui utilise le document XML des informations.

Exemple:

Afin d'afficher les données d'un document XML dans un navigateur, on peut lui attacher une feuille de style CSS.

```
<?xml-stylesheet type="text/css" href="cours.css"?>
```

Les commentaires XML sont les mêmes qu'en HTML (ceci est dû au lien de parenté avec SGML). Ils se positionnent n'importe où après le prologue et peuvent figurer sur plusieurs lignes.

```
<!-- Date de création : 12/10/09 -->
```

2.3 Les nœuds élément

Les éléments gèrent la structuration des données d'un document XML, un peu à la manière des répertoires qui servent à l'organisation des fichiers. On peut les qualifier de métadonnées, au sens où ils ne font pas partie réellement des données mais servent à en désigner la nature. À la place du terme élément, on peut utiliser les termes balise, tag ou encore nœud.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<cours>
  <intervenant>
    NFAOUI
  </intervenant>
  <separateur/>
  <chapitre>
    Formation JEE
    <para>Un paragraphe</para>
    <para>Autre paragraphe</para>
  </chapitre>
</cours>
```

2.4 Les attributs d'un élément

Un attribut est un couple (clé, valeur) associé à la définition d'un élément. Il est localisé dans la balise ouvrante de l'élément. Un élément peut donc avoir de 0 à n attributs uniques. L'attribut est complémentaire de l'élément de par son rôle au sens où il ajoute une information à l'élément ou bien encore le complète dans sa définition.

Exemple :

```
<auteur nom="NFAOUI" prenom="El Habib">...</auteur>  
<contact email='nfhabib@yahoo.fr'>/>
```

- **nom** et **prenom** sont des attributs de l'élément **auteur** alors que **email** est un attribut de l'élément **contact**.
- On utilise soit les **guillemets**, soit les **apostrophes** pour encapsuler les valeurs.

2.5 Choix entre éléments et attributs

Ce qui s'écrit avec des attributs peut également l'être en s'appuyant uniquement sur des éléments.

Exemple :

Cas avec attributs :

```
<personne nom="NFAOUI" prenom="El Habib"/>
```

Cas sans attribut :

```
<personne>  
  <nom>  
    NFAOUI  
  </nom>  
  <prenom>  
    El Habib  
  </prenom>  
</personne>
```

On peut définir cependant quelques règles simples pour déterminer s'il est préférable d'utiliser un attribut ou un élément. Lorsqu'une valeur est de taille modeste, a peu de chance d'évoluer vers une structure plus complexe, et n'est pas répétée, alors

l'attribut peut tout à fait convenir. Dans tous les autres cas, l'élément reste incontournable. Si vous avez un doute entre un attribut et un élément, alors choisissez toujours l'élément qui est le plus ouvert et garantira le plus facilement une évolution.

En résumé :

- Quand utiliser les attributs ?
Valeur unique de type simple (information monovaluée)
- Quand utiliser les éléments ?
Valeur de type complexe (énumérations, possède des propriétés)

2.6 Les nœuds textes

Dans un document XML, ce qui est appelé donnée est le texte qui est associé à l'attribut, c'est-à-dire sa valeur, ou à l'élément, c'est-à-dire son contenu. Les données constituent le cœur du document, et tout le reste, le formalisme, ne sert qu'à séparer et classer ces données. Celles-ci sont dites terminales dans l'arborescence XML, dans la mesure où il ne peut y avoir de structuration supplémentaire. Dans le vocabulaire propre à DOM (Document Object Model), que nous aborderons dans un autre chapitre, la donnée apparaît comme un **nœud texte**. Elle est bien entendu liée au prologue puisque l'encodage sert à désigner le jeu de caractères utilisé dans votre document.

```
<cours titre="XML">
  <intervenant nom="NFAOUI">
  </intervenant>
  <plan>
    Introduction
    XML au cœur des SI
  </plan>
</cours>
```

2.7 Les entités prédéfinies

Comme certains caractères sont réservés à la syntaxe XML, il faut être vigilant lors de l'écriture des données. Pour cela, nous disposons **d'entités prédéfinies**. L'entité en elle-même peut être perçue comme un raccourci vers une autre valeur, ce qui a l'avantage de faciliter la maintenance (changement de valeur avec répercussions) et de masquer les conflits syntaxiques.

Voici la liste des entités prédéfinies :

- < équivalent de < (*less than*) ;
- > équivalent de > (*greater than*) ;
- & équivalent de & (*ampersand*) ;
- " équivalent de " (*quote*) ;
- ' équivalent de ' (*apostrophe*).

Exemple :

Dans un élément <calcul>, le texte « if (a<b et b>c) » peut être écrit correctement comme le suivant:

```
<calcul>
  If (a&lt;b et b&gt;c)
</calcul>
```

Bien entendu, les entités prédéfinies ne servent qu'à lever une ambiguïté syntaxique pour le parseur. Ces entités peuvent être utilisées dans un élément, pour du texte, ou dans un attribut, pour sa valeur.

2.8 Les sections CDATA (Character Data)

Les entités prédéfinies présentes en trop grand nombre dans un même bloc peuvent alourdir inutilement le document. Dans le cas du contenu textuel d'un élément (et uniquement dans ce cas), nous disposons des sections CDATA (*Character Data*). Cette section doit être considérée comme un bloc de texte dont les caractères seront pris tel quel par le parseur jusqu'à la séquence de fin]]>.

Exemple :

```
<![CDATA[
<element>C'est un document XML
</element>
]]>
```

Dans cet exemple, <element> n'est pas considéré comme une balise de structuration, mais comme du texte.

2.9 Quelques règles de syntaxe

Ces règles de syntaxe sont à respecter impérativement pour qu'un document XML soit bien formé.

- Le nom d'un élément ne peut commencer par un chiffre.
- Si le nom d'un élément est composé d'un seul caractère il doit être dans la plage [a-zA-Z] (c'est-à-dire une lettre minuscule ou majuscule sans accent) ou _ ou :.
- Avec au moins 2 caractères, le nom d'un élément peut contenir _, -, . et : plus les caractères alphanumériques (attention, le caractère : est réservé à un usage avec les espaces de nom que nous aborderons par la suite).
- Tous les éléments ouverts doivent être fermés.
- Un élément parent est toujours fermé après la fermeture des éléments fils. Voici un contre-exemple où l'élément fils b est incorrectement fermé après la fermeture de son élément parent a : `<a><b></a></b>`.

Pour connaître plus précisément les caractères autorisés, vous pouvez vous reporter à la grammaire XML du W3C disponible à l'adresse

<http://www.w3.org/TR/2006/REC-xml-20060816/#sec-well-formed>.

2.10 Quelques conventions de nommage

Voici quelques conventions souvent employées dans les documents XML :

- Employer des minuscules pour les attributs et les éléments.
- Éviter les accents dans les noms d'attributs et d'éléments pour des raisons de compatibilité avec les outils du marché qui proviennent souvent d'un univers anglo-saxon.
- Préférer les **guillemets** délimitant les valeurs d'attribut.
- Séparer les noms composés de plusieurs mots par les caractères -, _, . ou une majuscule.
Essayer d'être homogène dans votre document en gardant la même convention.

Exemple :

```
<value-of/>  
<valEntier></valEntier>
```

2.11 Quelques exemples XML

Voici quelques exemples simples de documents XML. Certaines formes de structuration d'usage courant ont été normalisées par différents organismes comme le W3C ou le consortium OASIS (<http://www.oasis-open.org>).

- ❑ **MathML** est une recommandation du W3C (<http://www.w3.org/Math/>) servant à d'écrire des expressions mathématiques.

Exemple:

Pour décrire l'expression : x^2+4x+4 en utilisant MathML,

```
<?xml version="1.0"?>
<math>
<mrow>
<msup> <mi>x</mi><mn>2</mn> </msup>
<mo>+</mo>
<mrow>
<mn>4</mn><mo>&InvisibleTimes;</mo><mi>x</mi>
</mrow>
<mo>+</mo>
<mn>4</mn>
</mrow>
</math>
```

- ❑ **VoiceXML** : est un standard pour décrire des choix (processus de questions/réponses) liés à des répondants vocaux (<http://www.voicexml.org/>).

3. Les espaces de noms

Pour délimiter la portée d'une balise, d'un attribut ou d'une valeur d'attribut, nous disposons d'espaces de noms (namespace). Par analogie, cette notion est reprise dans la plupart des langages de programmation récents, comme les packages en java ou les namespaces dans l'environnement .net. La notion d'espace de noms peut être perçue comme un groupe d'appartenance ou une famille. L'utilisation des espaces de noms garantit une forme de traçabilité de la balise et évite les ambiguïtés d'usage.

Un espace de noms XML est une collection de noms, identifiée par une référence d'URI, qui sont utilisés dans les documents XML comme types d'élément et d'attribut.

L'attribut **xmlns** (**ns** est pour *namespace*) sert de déclaration d'espace de noms. L'espace de noms est valable sur l'élément où il a été déclaré ainsi que sur les descendants de cet élément. L'espace de noms est souvent déclaré dans l'élément racine pour être valable sur tout le document.

Un préfixe est une sorte de raccourci vers l'URL de l'espace de noms. La notion de préfixe n'a de sens que par rapport à l'URL associée. Dans la pratique, l'application ne voit pas ce préfixe mais uniquement l'URL associée à telle ou telle partie du document. L'URI ne signifie pas qu'il doit y avoir un document sur votre serveur HTTP. Ce n'est qu'un identifiant et n'importe quelle chaîne de caractères pourrait en réalité être employée.

Exemple d'utilisation d'espace de noms

L'exemple ci-dessous montre que les éléments *fullName*, *mailbox* et *personalTitle* appartiennent à l'espace de noms identifié par l'URI « <http://www.w3.org/2000/10/swap/pim/contact> » et ayant comme préfixe « *contact* ». Alors que l'élément *Description* et l'attribut *about* appartiennent à l'espace de noms identifié par l'URI « <http://www.w3.org/1999/02/22-rdf-syntax-ns> » et ayant comme préfixe « *rdf* ».

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:contact="http://www.w3.org/2000/10/swap/pim/contact#" >
  <rdf:Description rdf:about="http://www.nfaoui.com/ma/">
    <contact:fullName>Ahmed RAMADANI</contact:fullName>
    <contact:mailbox rdf:resource="mailto:nfaoui@gmail.com" />
    <contact:personalTitle>Professeur</contact:personalTitle>
  </rdf:Description>
</rdf:RDF>
```

Chapitre 2

DTD (Document Type Definition)

1. Introduction

Une DTD permet de définir le «vocabulaire» et la structure qui seront utilisés dans le document XML. Elle permet de déclarer la liste, le type et les relations des éléments, des attributs, des entités et des notations contenus dans le document XML.

La DTD contraint le document XML à se conformer aux règles énoncées. Si le document est conforme au DTD, alors ce document XML est considéré comme *valide*.

Chaque élément, attribut et entité doivent être déclarés dans la DTD avant leur utilisation.

2. Déclaration de type de document

La **déclaration de type de document** permet d'associer une DTD à un document XML. Elle doit se situer au début du document XML, juste après la déclaration XML. Lorsque vous associez une DTD à un document, votre **déclaration de type de document** peut suivre deux approches:

- Elle peut directement inclure des déclarations de balisage dans le document, qui constituent ainsi la **DTD interne**.
- Elle peut référencer des déclarations de balisage externes, qui constituent la **DTD externe**.

Syntaxe générale d'une déclaration de type de document :

```
<?xml version="1.0" standalone="no" ?>  
<!DOCTYPE RootElem SYSTEM ExrenalDTDRef [InternalDTDDDecl]>
```

RootElem : identifie l'élément racine de la classe de documents.

SYSTEM : indique que la DTD est située dans un fichier externe.

ExrenalDTDRef: URI (Uniform Resource Identifier) d'un fichier qui contient la DTD externe.

La DTD interne correspond à *InternalDTDRef* et est déclarée entre crochets ([]).

L'état autonome (standalone) d'un document se trouvant dans la déclaration XML détermine si un document repose ou non sur une quelconque source d'informations externe, comme une DTD externe.

standalone="yes": document autonome ne dépendant pas de sources d'informations externes.

standalone="no": Valeur par défaut. Elle indique que le document n'est pas autonome. Par exemple, un document qui repose sur une DTD externe pour sa validation.

Exemple :

```
<?xml version="1.0" standalone="no" ?>
<!DOCTYPE jeuhistoire SYSTEM "jeuhistoire.dtd" [
<!ELEMENT question (#PCDATA)> ]>
```

DTD interne ou DTD externe ?

- ❑ Il faut privilégier les DTD internes pour des petits projets dont les données ne se trouvent que dans ce seul et unique document.
- ❑ Pour des projets plus importants pour lesquels plusieurs ou de nombreux documents se réfèrent à une DTD commune, vous devez créer une DTD externe.
- ❑ Les déclarations de balisage internes (DTD interne) outrepassent toujours les déclarations externes (DTD externe) qui portent le même nom.

3. Bases de la construction de DTD

Une DTD permet à l'aide de **déclarations de balisage** de déclarer la liste, le type et les relations des éléments, des attributs, des entités et des notations contenus dans le

document XML. Les différents types de déclarations de balisage pouvant être employés dans une DTD sont :

- Les éléments autorisés dans le document;
- Les attributs qui peuvent être affectés à chaque élément;
- Les entités autorisées dans le document;
- Les notations autorisées à employer avec des entités externes.

3.1 Déclaration d'un élément

`<! ELEMENT nomElement Type >`

- *nomElement*: détermine le nom de la balise employée pour le baliser dans un document. Il doit être unique dans le contexte d'une DTD.
- *Type*: spécifie le type de l'élément. XML prend en charge quatre types différents d'éléments, déterminés par le contenu de cet élément:
 - ❑ EMPTY : L'élément est vide. Il ne possède aucun contenu mais peut posséder des attributs.
 - ❑ Elements uniquement: l'élément ne contient que des éléments enfants.
 - ❑ MIXED: l'élément contient à la fois des éléments enfants et des données caractères.
 - ❑ ANY : L'élément renferme tout contenu autorisé par la DTD.

Note :

Le nom d'un élément ne peut contenir d'esperluette (&) ni commencer par la séquence de lettres XML dans n'importe quelle casse (XML, xml, Xml, etc.)

3.1.1 Élément vide

Exmple 1 : élément vide sans attributs

Déclaration :

`<!ELEMENT habit EMPTY>`

Utilisation :

`<habit />`

Ou (paire de balises d'ouverture et de fermeture)

`<habit></habit>`

Exemple2 : élément vide avec attributs

`<habit type="t-shirt" couleur="navy" taille="xl" />`

3.1.2 Élément composé d'une suite d'éléments (éléments uniquement)

Il ne contient que des éléments enfants. Autrement dit, il ne renferme jamais de contenu texte.

Syntaxe générale :

`<!ELEMENT NomElement ModèleContenu>`

ModèleContenu :

Énumère les éléments enfants qui peuvent être présents dans l'élément. Le modèle de contenu est spécifié à l'aide d'une combinaison de symboles spéciaux de déclaration d'éléments et des noms des éléments enfants.

Exemples :

Déclaration d'un élément nommé **curriculum** qui pourrait servir dans un langage de balisage de curriculum vitae :

`<!ELEMENT curriculum (intro, (formation | experience+)+, loisirs?, references*)>`

Le premier élément enfant est **intro**, qui doit apparaître une fois et une seule dans l'élément curriculum, puisque aucun symbole ne lui est associé.

Un bloc d'éléments **formation** et **experience** doit ensuite apparaître au moins une fois, comme l'indique le signe + à l'extérieur des parenthèses.

L'élément **formation** doit apparaître une fois et une seule, tandis que

l'élément **expérience** doit apparaître au moins une fois.

L'élément **loisirs** peut apparaître zéro ou une seule fois.

L'élément **references** peut être présent un nombre quelconque de fois.

3.1.3 Élément mixte

Un élément mixte peut renfermer à la fois des données caractères (du texte) et des éléments enfants.

Le plus simple des éléments mixtes est connu sous le nom **d'élément en texte seul**. Un tel élément est déclaré comme suit :

```
<!ELEMENT NomElement (#PCDATA)>
```

PCDATA (Parsed Character DATA) fait référence au fait que les PCDATA d'un document sont traités (parsed) par une application XML.

Exemple :

Déclaration :

```
<!ELEMENT loisirs (#PCDATA)>
```

Utilisation :

```
<loisirs>jonglage, monocycle, natation</loisirs>
```

Le modèle de contenu d'un élément mixte doit contenir une liste de choix qui se présente comme suit :

```
<!ELEMENT NomElement (#PCDATA | ListeElement)*>
```

Un astérisque (*) doit être placé à la fin de la liste de choix du modèle de contenu pour indiquer que la totalité du groupe de choix est facultative: c'est une exigence des éléments mixtes.

3.1.4 Élément quelconque

Il est déclaré à l'aide du symbole ANY et peut contenir n'importe quel type d'élément, donnée caractère ou mélange des deux. En quelque sorte, c'est un élément mixte dépourvu de toute contrainte sur son modèle de contenu.

Syntaxe :

<!ELEMENT *NomElement* ANY>

Le manque de structure associé aux éléments quelconques fait que vous devriez les éviter à tout prix dans une DTD de production.

Ils sont classiquement employés aux fins de test pendant la phase de développement d'une DTD, puis supprimés dans la DTD de production terminée.

Exemple :

Déclaration :

```
<!ELEMENT personne (nom, prenom+, tel?, adresse) >
  <!ELEMENT nom (#PCDATA) >
  <!ELEMENT prenom (#PCDATA) >
  <!ELEMENT tel(#PCDATA) >
  <!ELEMENT email (#PCDATA) >
  <!ELEMENT adresse (ANY) >
```

Utilisation :

```
<personne>
  <nom>Hugo</nom>
  <prenom>Victor</prenom>
  <prenom>Charles</prenom>
  <tel>01120243</tel>
  <adresse><rue></rue><ville>Paris</ville></adresse>
</personne>
```

3.2 Déclaration des attributs d'un élément

A chaque type d'élément est attaché un ensemble d'attributs.

Une définition d'attributs a la forme suivante :

<!ATTLIST *NomElement*

NomAttribut type Default ...
NomAttribut type Default>

Où :

- Le *type* est celui des valeurs de l'attribut,
- *Default* spécifie si la valeur de l'attribut doit être ou non présente dans le document et fournit éventuellement une valeur par défaut.

Les noms d'attributs sont locaux à chaque type d'élément : deux éléments de types différents peuvent avoir des attributs de même nom.

3.2.1 Valeur par défaut (Default)

La valeur par défaut fait référence à une valeur ou à un symbole qui indique l'emploi de l'attribut.

- **#REQUIRED**: l'attribut est obligatoire. Il doit être présent dans la balise de l'élément
- **#IMPLIED**: l'attribut est facultatif
- **#FIXED valeur**: l'attribut possède une valeur fixe.
- **valeur**: valeur à affecter à l'attribut s'il est absent de la balise de l'élément (valeur par défaut),

Exemple :

```
<!ELEMENT distance (#PCDATA)>  
<!ATTLIST distance unites (miles | kilometres | tours) "miles" >
```

3.2.2 Types d'attributs

Les types d'attributs peuvent être répartis en trois groupes : chaînes, énumérations et jetons.

- Les attributs **chaînes** sont représentés par le type : CDATA (texte)

Exemple :

```
<!ATTLIST formation etablisement CDATA #IMPLIED >
```

- ❑ Le type **énumération** est analogue au type CDATA à l'exception près que les valeurs acceptables doivent provenir d'une liste présente dans la déclaration de liste d'attribut.

Exemple :

```
<!ATTLIST formation diplôme (lyceen | baccalaureat | maitrise |  
doctorat ) "baccalaureat" >
```

- ❑ Les attributs **jetons** sont traités comme des jetons (token) par une application XML, ce qui signifie que celle-ci convertit tous les espaces vides en un unique espace et élimine les espaces vides de début et de fin d'une valeur d'attribut jeton. En outre, l'application XML valide la valeur d'un attribut jeton d'après le type déclaré : ENTITY, ENTITIES, ID, IDREF, IDREFS, NMTOKEN, NMTOKENS ou NOTATION.

- Le type **ENTITY** sert à référencer une entité externe binaire. Le type **ENTITIES** permet de spécifier plusieurs entités externes binaires séparées par des espaces.

Exemple :

Les images sont classiquement référencées comme entités binaires

```
<!ATTLIST photo image ENTITY #IMPLIED>
```

- Le type ID est un identifiant unique qui peut être employé pour identifier de façon unique un **élément** d'un document.

Exemple:

```
<!ATTLIST partie id ID #REQUIRED>
```

- IDREF: une référence vers un ID déclaré quelque part ailleurs dans la DTD.
- IDREFS: plusieurs références vers des ID déclarés quelques part ailleurs dans le document.
- NMTOKEN ou NMTOKENS : token de nom ou suite de tokens de nom. Ces deux types servent à spécifier des attributs qui contiennent des valeurs de noms jeton (token). Une valeur de nom jeton se compose d'un nom simple (qui ne peut

renfermer). Plus spécifiquement, une valeur de nom jeton peut se composer de caractères alphanumériques ainsi que de caractères ., -, _ et :.

3.2.3 Exemple d'attributs

```
<! ATTLIST personne  
    num ID,  
    age CDATA,  
    genre (Masculin | Feminin ) >
```

```
<!ELEMENT auteur (#PCDATA) >
```

```
<!ELEMENT editeur (#PCDATA) >
```

```
<!ATTLIST auteur  
    genre (Masculin | Feminin ) #REQUIRED  
    ville CDATA #IMPLIED>
```

```
<!ATTLIST editeur  
    ville CDATA #FIXED "Casa">
```

Chapitre 3

XML Schema

1. Insuffisance des DTD

Les DTD souffrent de lacunes comme :

- Pas de types de données à part du texte (#PCDATA)
- Syntaxe spécifique (pas XML)
 - difficile à interpréter
 - difficile à traduire en schéma objets

Les schémas XML (XML-schema du W3C) permettent de combler ces lacunes.

2. XML Schema : Définitions de base

Un schéma d'un document définit :

- les éléments possibles dans le document
- les attributs associés à ces éléments
- la structure du document
- les types de données

Le schéma est spécifié en XML :

- pas de nouveau langage
- balisage de déclaration
- espace de nom spécifique xsd: ou xs:

Il présente de nombreux avantages

- structures de données avec types de données
- extensibilité par héritage et ouverture
- analysable à partir d'un parseur XML standard

3. Bases de la construction de XML Schema

Le langage qui sert à décrire les langages de balisage en XML Schema est XSD (XML Schema Definition). Les schémas créés avec ce langage sont souvent simplement nommés XSD.

Le langage XSD est fondé sur XML. Cela signifie que vous employez des éléments et attributs XML pour décrire la structure de votre propre langage de balisage personnalisé.

La structure générale d'un document de schéma XSD est la suivante:

```
<?xml version="1.0"?>
  <xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  </xsd:schema>
```

Commentaires:

- Comme un document de schéma XSD n'est qu'un document XML, vous devez placer en tête la classique déclaration XML
`<?xml version="1.0"?>`
- L'élément racine d'un document XSD se nomme **xsd:schema**.
- L'attribut `xmlns:xsd` sert à définir l'espace de noms XSD. L'espace de noms associé à XML Schema est :

<http://www.w3.org/2001/XMLSchema> (URI standard rendue disponible par le W3C) dont le préfixe pourra être nommé de façon quelconque. Mais, le préfixe usuel adopté par les développeurs XML est `xsd` ou parfois `xs`. Vous devez donc faire précéder le nom de chaque élément et attribut par **xsd:** ou **xs:**.

3.1 Types de données XSD

Il existe deux grands types de données en XSD : les données simples et les données complexes. Une donnée simple correspond à une information basique comme un nombre, une chaîne de caractères, une date, une heure, une liste, etc. En revanche, une donnée complexe représente des informations plus évoluées comme des éléments mixtes et des séquences d'éléments.

Les types de données simples peuvent être employés tant dans des éléments que dans des attributs.

- ❑ L'élément **xsd:element** sert à créer un élément de type simple.
- ❑ L'élément **xsd:attribute** sert à créer des attributs.

Exemple 1 :

```
<?xml version="1.0" encoding="UTF-8"?>
  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="nom" type="xs:string"/>
    <xs:element name="titre" type="xs:string"/>
    <xs:element name="fonction" type="xs:string"/>
    <xs:attribute name="datenaissance" type="xs:date"/>
    <xs:attribute name="poids" type="xs:integer"/>
  </xs:schema>
```

Cet exemple montre comment fonctionne les types de données simples avec les éléments et les attributs *sans toutefois dévoiler les relations entre éléments et attributs*. Ces relations sont établies à l'aide de types de données complexes, capables de détailler les modèles de contenu des éléments.

Ci-après un exemple (exemple 2) d'emploi de types de données simples dans un type complexe pour décrire le modèle de contenu d'un élément nommé **personne**.

Exemple 2 :

```
<?xml version="1.0" encoding="UTF-8"?>
  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="personne">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="nom" type="xs:string"/>
          <xs:element name="titre" type="xs:string"/>
          <xs:element name="fonction" type="xs:string"/>
        </xs:sequence>
        <xs:attribute name="datenaissance" type="xs:date"/>
        <xs:attribute name="poids" type="xs:integer"/>
      </xs:complexType>
    </xs:element>
  </xs:schema>
```

L'élément `personne` possède deux attributs, `datenaissance` et `poids`, ainsi que trois éléments enfants : `nom`, `titre` et `fonction`.

Exemple de document XML qui suit la structure définie par le XSD de l'exemple 2:

```
<personne datenaissance="10-10-1980" poids="65">
  <nom>Salim</nom>
  <titre>Mr.</titre>
  <fonction>Policier</fonction>
</personne>
```

3.2 Schémas XSD et documents XML

Pour associer un schéma à un document XML aux fins de validation, vous fixez l'attribut **`noNamespaceSchemaLocation`** de l'élément racine à l'emplacement du document de schéma. Vous devez toutefois déclarer l'espace de noms auquel appartient cet attribut préalablement à son emploi.

Exemple :

```
<?xml version="1.0" encoding="UTF-8"?>
<contacts xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="contacts.xsd">
  <personne datenaissance="10-10-1980" poids="65">
    <nom>Salim</nom>
    <titre>Mr.</titre>
    <fonction>Policier</fonction>
  </personne>
</contacts>
```

3.3 Types simples

XSD définit plusieurs types simples comme :

- `string`, `byte`, `integer`, `positiveInteger`, `negativeInteger`, `int`
- `boolean`
- `date`

Et beaucoup d'autres : short, long, float

3.4 Détails des types complexes

Tous les types complexes sont créés à l'aide de l'élément **xsd:complexType**. Cet élément possède un attribut nommé **name** qui sert à nommer un type complexe. Vous pouvez également déclarer directement un type complexe au sein d'un élément. Dans ce cas-là, un nom n'est pas nécessaire.

Les types complexes peuvent être répartis en quatre catégories principales:

- ☐ éléments vides;
- ☐ élément éléments seulement (Élément composé d'une suite d'éléments);
- ☐ éléments mixtes;
- ☐ séquences et choix

3.4.1 Éléments vides

Un élément vide ne contient ni texte ni élément enfant, mais peut posséder des attributs.

Vous créez un élément vide à l'aide de l'élément **xsd:complexType** en combinaison avec l'élément **xsd:complexContent**.

Exemple :

```
<xsd:element name="automobile">
  <xsd:complexType>
    <xsd:complexContent>
      <xsd:extension base="xsd:anyType">
        <xsd:attribute name="code" type="xsd:string"/>
        <xsd:attribute name="annee" type="xsd:gYear"/>
        <xsd:attribute name="constructeur" type="xsd:string"/>
        <xsd:attribute name="modele" type="xsd:string"/>
      </xsd:extension>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:element>
```

Emploi de l'élément automobile dans un document XML :

```
<automobile code="SALHV1245SA661555" annee="2003"
constructeur="Land Rover" modele="Range Rover" />
```

3.4.2 Séquences et choix

Un aspect puissant des types complexes est leur capacité à organiser les éléments en séquences et choix. Une séquence est une liste d'éléments enfants qui doivent apparaître dans un ordre donné, tandis qu'un choix est une liste d'éléments enfants dont un seul doit être employé.

L'élément **xsd:sequence** permet de créer une séquence.

Exemple :

```
<xsd:element name="quiz">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="question" type="xsd:string" />
      <xsd:element name="reponse" type="xsd:string" />
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Dans cet exemple, l'élément **quiz** contient deux éléments enfants, **question** et **reponse**, qui doivent apparaître dans l'ordre spécifié. Par défaut, une séquence ne doit être présente qu'une fois dans un élément. On peut recourir aux attributs **xsd:minOccurs** et **xsd:maxOccurs** pour que la séquence puisse intervenir plusieurs fois. L'attribut **xsd:maxOccurs** peut être fixé à la valeur **unbounded** pour autoriser un nombre illimité de séquences.

```
<xsd:sequence minOccurs="1" maxOccurs="20">
```

Exemple de document XML utilisant l'élément **quiz** :

```
<quiz>
  <qusetion>Que signifie XML ?</qusetion>
  <reponse>eXtensible Markup Language</reponse>
  <qusetion>Qui est responsable du suivi de XML ?</qusetion>
```

```
<reponse>World Wide Web Consortium (W3C)</reponse>
</quiz>
```

Un choix permet d'énumérer plusieurs éléments enfants et/ou séquences, un seul d'entre eux étant autorisé dans tout élément donné.

Vous créez un choix à l'aide de l'élément **xsd:choice**, qui contient la liste des éléments du choix.

Exemple :

```
<xsd:element name="id">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="noSecu" type="xsd:string" />
      <xsd:sequence>
        <xsd:element name="nom" type="xsd:string" />
        <xsd:element name="dateNaiss" type="xsd:date" />
      </xsd:sequence>
      <xsd:element name="noPermis" type="xsd:string" />
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

Exemple d'utilisation de l'élément **id**:

```
<id>
  <noSecu>1234556438</noSecu>
</id>

<id>
  <nom>Salim Salmoun</nom>
  <dateNaissance>1970-10-28</dateNaissance>
</id>

<id>
  <noPermis>13489765</noPermis>
</id>
```

3.4.3 Élément éléments seulement

Un élément **éléments seulement** ne contient que des éléments enfants, mais pas de texte. Il peut bien sûr toutefois également posséder des attributs.

L'élément **xsd:complexType** sert à créer ce type d'élément.

Exemple :

```
<xsd:element name="quiz">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="question" type="xsd:string" />
      <xsd:element name="reponse" type="xsd:string" />
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

3.4.4 Éléments mixtes

Un élément mixte contient à la fois du texte et des éléments enfants.

Les éléments à texte seul sont considérés comme un type d'élément mixte et ne peuvent contenir que du texte, sans élément enfant.

Vous pouvez créer des éléments texte seul à l'aide de l'élément **xsd:complexType** combiné avec l'élément **xsd:simpleContent**.

Un élément mixte est créé comme tout autre type complexe, mais complété avec l'attribut **xsd:mixed**.

Exemple : Élément texte seul

```
<xsd:element name="distance">
  <xsd:complexType>
    <xsd:simpleContent>
      <xsd:extension base="xsd:decimal">
        <xsd:attribute name="unite" type="xsd:string" use="required"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:element>
```

Exemple d'utilisation de l'élément **distance** :

```
<distance unite="miles">12.5</distance>
```

Exemple : Élément mixte

```
<xsd:element name="message">
  <xsd:complexType mixed="true">
    <xsd:sequence>
      <xsd:element name="emph" type="xsd:string" />
    </xsd:sequence>
    <xsd:attribute name="A" type="xsd:string" use="required" />
    <xsd:attribute name="De" type="xsd:string" use="required" />
    <xsd:attribute name="date" type="xsd:time" use="required" />
  </xsd:complexType>
</xsd:element>
```

Exemple d'utilisation :

```
<message A="toi" De="moi" date="2005-11-24 14:45:00" />
Vas-tu bientôt revenir ? Tu me manques <emph>vraiment</emph>!
</message>
```

Chapitre 4

Exercices corrigés

1. Série d'exercices 1 : Structuration des données en XML

Exercice 1 :

On se propose d'écrire une version structurée en XML du document suivant (les valeurs présentées ici sont à titre d'exemple. Elles ne sont pas réelles) :

Une bouteille d'eau AinImouzer de 150 cl contient par litre 71 mg d'ions positifs calcium, et 5,5 mg d'ions positifs magnésium. On y trouve également des ions négatifs comme des chlorures à 20 mg par litre et des nitrates avec 1 mg par litre. Elle est recueillie à Imouzer la Source, dans le département de Fes-Boulemane. Son code barre est 3274080005006 et son pH est de 7,45. Comme la bouteille est sale, quelques autres matériaux comme du fer s'y trouvent en suspension.

Une seconde bouteille d'eau EauHammat a été, elle, recueillie à la source d'Ain Hammat dans les régions de Tafilalet. La concentration en ions calcium est de 98 mg/l, et en ions magnésium de 4 mg/l. Il y a 3,6 mg/l d'ions chlorure et 2 mg/l de nitrates, pour un pH de 7,4. Le code barre de cette bouteille de 50 cl est 3268840001003.

Une bouteille de même contenance est de marque SidiSami, et a été puisée à la source MoulaySami de SoussMassaDaraa, bien connu pour ses sources donnant un pH neutre de 7. Elle comprend 11,5 mg/l d'ions calcium, 8,0 mg/l d'ions magnésium, 13,5 mg/l d'ions chlorures et 6,3 mg/l d'ions nitrates. Elle contient également des particules de silice. Son code barre est 3057640117008.

- Proposer une représentation graphique d'un arbre XML pour ce document.
- Proposez une structuration XML de ce document.
- Vérifiez, à l'aide de l'éditeur, que votre document est bien formé.

Exercice 2 : Structuration du contenu d'un livre en XML

On se propose de **structurer** le contenu d'un livre en utilisant le formalisme XML. Le livre est structuré en sections (au moins 2), en chapitres (au moins 2) et en paragraphes (au moins 2).

Le livre doit contenir la liste des auteurs (avec nom et prénom).

Tous les éléments doivent posséder un titre, sauf le paragraphe qui contient du texte.

Remarque :

- Dans cette première version, nous ne voulons pas utiliser d'attributs ;
 - L'encodage utilisé est ISO-8859-1
- a. Proposer une structuration XML de ce document (avec 2 auteurs, 2 sections, 2 chapitres par section et 2 paragraphes par chapitre). Vous nommez le document « *livre1.xml* ».
 - b. Vérifier, à l'aide de l'éditeur, que votre document est bien formé.

Exercice 3 : Utilisation des attributs

On souhaite compléter la structure du document XML de l'exercice précédent par les attributs *nom* et *prenom* pour les auteurs et *titre* pour le livre, les sections et les chapitres.

- a. Analysez la structure du nouveau document. Y a-t-il des simplifications possibles ?
- b. Placez dans 2 paragraphes un bloc de texte contenant l'extrait suivant :
`<element id="10">></element>`

Pour le premier paragraphe, employez les entités prédéfinies.

Pour le deuxième paragraphe, employez une section CDATA.

- c. Vérifiez, à l'aide de l'éditeur, que votre document est bien formé.

Exercice 4 :

On se propose de développer un jeu de culture générale en informatique nommé **JeuInfo**. L'idée consiste à employer des documents XML pour procurer les questions aux versions Web et mobiles du jeu. Ainsi, nous avons besoin de coder la base de données des questions et des réponses en XML. Le recours à XML permet

de coder le contenu d'une façon cohérente et de fournir facilement et régulièrement des mises à jour et de nouveaux contenus.

Le jeu implique un seul type de question à choix multiples. Chaque question possède plusieurs choix de réponses possibles, un ou plusieurs choix à la fois peuvent être corrects (valides).

- Proposer une représentation graphique d'un arbre XML pour le contenu de ce jeu.
- Proposer un document XML **JeuInfo.xml** comprenant trois questions.
- Associer une feuille de style CSS "**JeuInfo.css**" au document **JeuInfo.xml** pour visualiser les données sur un navigateur Web.
- Vérifier, à l'aide de l'éditeur, que le document est bien formé.

Exercice 5 : Le format MathML

MathML est une recommandation du W3C (<http://www.w3.org/Math/>) servant à décrire des expressions mathématiques. Quelle est l'expression décrite par le document MathML suivant ?

```
<?xml version="1.0" encoding="UTF-8"?>
<math >
  <mrow>
    <mrow>
      <msup> <mi>x</mi> <mn>2</mn> </msup> <mo>+</mo>
      <mrow>
        <mn>4</mn>
        <mo>&InvisibleTimes;</mo>
        <mi>x</mi>
      </mrow>
      <mo>+</mo>
      <mn>4</mn>
    </mrow>
    <mo>=</mo>
    <mn>0</mn>
  </mrow>
</math>
```

2. Série d'exercices 1 : Corrigé

Exercice 1 :

N.B. Le PH est une caractéristique de l'eau et non pas de la bouteille. Il est préférable de le prendre comme élément.

Structuration XML du document

```
<?xml version="1.0" encoding="UTF-8"?>
<documentEau>
  <bouteille>
    <marque> AinImouzer</marque>
    <contenance unite="cl"> 150</contenance>
    <composition>
      <ion type="positif" >
        <nom> calcium</nom>
        <concentration unite="mg/l">71</concentration>
      </ion>
      <ion type="positif">
        <nom> magnesium</nom>
        <concentration unite="mg/l"> 5,5</concentration>
      </ion>
      <ion type="negatif">
        <nom>chlorures </nom>
        <concentration unite="mg/l"> 20</concentration>
      </ion>
      <ion type="negatif">
        <nom>nitrates </nom>
        <concentration unite="mg/l"> 1</concentration>
      </ion>
      <autre type="metal">fer</autre>
    </composition>
    <source>
      <nom>Imouzer la Source</nom>
      <departement> Fes-Boulemane</departement>
    </source>
    <ph>7,45</ph>
    <code-bare>3274080005003</code-bare>
```

</bouteille>

<bouteille>

<marque> EauHammat</marque>

<contenance unite="cl"> 50</contenance>

<composition>

<ion type="positif">

<nom> calcium</nom>

<concentration unite="mg/l">98</concentration>

</ion>

<ion type="positif">

<nom> magnesium</nom>

<concentration unite="mg/l"> 4</concentration>

</ion>

<ion type="negatif">

<nom>chlorures </nom>

<concentration unite="mg/l">3,6 </concentration>

</ion>

<ion type="negatif">

<nom>nitrates </nom>

<concentration unite="mg/l"> 2</concentration>

</ion>

</composition>

<source>

<nom>Ain Hammat</nom>

<departement>Tafilalt</departement>

</source>

<ph>7,4</ph>

<code-bare>3268840001008</code-bare>

</bouteille>

<bouteille>

<marque> SidiSami</marque>

<contenance unite="cl"> 150</contenance>

<composition>

<ion type="positif">

<nom> calcium</nom>

<concentration unite="mg/l">11,5</concentration>

</ion>

<ion type="positif">

<nom> magnesium</nom>

```

        <concentration unite="mg/l"> 8,0</concentration>
    </ion>
    <ion type="negatif">
        <nom>chlorures </nom>
        <concentration unite="mg/l"> 13,5</concentration>
    </ion>
    <ion type="negatif">
        <nom>nitrates </nom>
        <concentration unite="mg/l">6,3</concentration>
    </ion>
    <autre type="">silice</autre>
</composition>
<source>
    <nom>MoulaySami</nom>
    <departement> SoussMassaDaraa</departement>
</source>
<ph>7</ph>
<code-bare>3057640117008</code-bare>
</bouteille>
</documentEau>

```

Exercice 2 :

a. Structuration XML du document

```

<?xml version="1.0" encoding="ISO-8859-1"? >
<livre>
    <titre>XML</titre>

    <auteurs>
        <auteur>
            <nom>NomAuteur</nom>
            <prenom>PrénomAuteur</prenom>
        </auteur>
        <auteur>
            <nom>NomAuteur</nom>
            <prenom>PrénomAuteur</prenom>
        </auteur>
    </auteurs>

```

```

<sections>
  <section>
    <titre>Section 1</titre>
    <chapitres>
      <chapitre>
        <titre>Chapitre 1</titre>
        <paragraphes>
          <paragraphe>Premier paragraphe</paragraphe>
          <paragraphe>Deuxième paragraphe</paragraphe>
        </paragraphes>
      </chapitre>
      <chapitre>
        <titre>Chapitre 2</titre>
        <paragraphes>
          <paragraphe>Premier paragraphe</paragraphe>
          <paragraphe>Deuxième paragraphe</paragraphe>
        </paragraphes>
      </chapitre>
    </chapitres>
  </section>
  <section>
    <titre>Section 2</titre>
    <chapitres>
      <chapitre>
        <titre>Chapitre 1</titre>
        <paragraphes>
          <paragraphe>Premier paragraphe</paragraphe>
          <paragraphe>Deuxième paragraphe</paragraphe>
        </paragraphes>
      </chapitre>
      <chapitre>
        <titre>Chapitre 2</titre>
        <paragraphes>
          <paragraphe>Premier paragraphe</paragraphe>
          <paragraphe>Deuxième paragraphe</paragraphe>
        </paragraphes>
      </chapitre>
    </chapitres>
  </section>
</sections>

```

</livre>

Exercice 3 :

a. Structuration XML du document

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<livre titre="XML et ses applications">
  <auteurs>
    <auteur nom="nom_auteur" prenom="pre_auteur"/>
    <auteur nom=" nom_auteur " prenom=" pre_auteur "/>
  </auteurs>
  <sections>
    <section titre="Section 1">
      <chapitre titre="Chapitre 1">
        <paragraphe>Premier paragraphe</paragraphe>
        <paragraphe>Deuxième paragraphe</paragraphe>
      </chapitre>
      <chapitre titre="Chapitre 2">
        <paragraphe>Premier paragraphe</paragraphe>
        <paragraphe>Deuxième paragraphe</paragraphe>
      </chapitre>
    </section>
    <section titre="Section 2">
      <chapitre titre="Chapitre 1">
        <paragraphe>Premier paragraphe</paragraphe>
        <paragraphe>Deuxième paragraphe</paragraphe>
      </chapitre>
      <chapitre titre="Chapitre 2">
        <paragraphe>Premier paragraphe</paragraphe>
        <paragraphe>Deuxième paragraphe</paragraphe>
      </chapitre>
    </section>
  </sections>
</livre>
```

b.

```
<?xml version="1.0" encoding="UTF-8"?>
<livre>
```

```

....
<chapitre titre="Chapitre1">
  <paragraphe>&lt;element
id="10"&gt;&amp;gt;&lt;/element&gt;</paragraphe>
  <paragraphe><![CDATA[<element
id="10">&gt;</element>]]></paragraphe>
</chapitre>
....
</livre>

```

Exercice 4 :

Structuration XML du document

```

<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/css" href="JeuInfo2.css"?>
<jeu-info>
  <question>
    <enonce> Quelle est la date de la première coupe du monde en foot ?
  </enonce>
    <choix num="1" isValid="true">1930</choix>
    <choix num="2" isValid="false">1920</choix>
    <choix num="3" isValid="false">1953</choix>
  </question>

  <question>
    <enonce>Quelle est la compagnie propriétaire de Windows ? </enonce>
    <choix num="1" isValid="false">InfoSoft</choix>
    <choix num="2" isValid="true">MicroSot</choix>
    <choix num="3" isValid="false">Oracle</choix>
  </question>>

  <question>
    <enonce>Quelle est la compagnie propriétaire de Dreamweaver ?
  </enonce>
    <choix num="1" isValid="false">Macromedia</choix>
    <choix num="2" isValid="true">Adobe</choix>

```



```
<choix num="3" isValid="false">Oracle</choix>
</question>
</jeu-info>
```

a. Feuille de style CSS : [JeuInfo_version2.css](#)

```
question {
  display: block;
  width: 750px;
  padding: 10px;
  margin-bottom: 10px;
  border: 4px double black;
  background-color: silver;
}

enonce {
  display: block;
  color: black;
  font-family: Times, serif;
  font-size: 16pt;
  text-align: left;
}

choix {
  display: block;
  color: maroon;
  font-family: Times, serif;
  font-size: 12pt;
  text-indent: 15px;
  text-align: left;
}
```

Exercice 5 :

Ce document MathML décrit l'expression suivante : $x^2 + 4x + 4 = 0$

Très brièvement, mo désigne un opérateur, mrow une expression, mi et mn respectivement des opérandes variable (par exemple x) et nombre.

3. Série d'exercices 2 : DTD (Document Type Definition)

Exercice 1 :

On se propose de concevoir une DTD pour un langage de balisage d'entraînement sportif. Ce langage de balisage qu'on va appeler ETML (Endurance Training Markup Language), pourrait être pratique si on envisage de participer à des compétitions de marathon ou de triathlon. Il modèle en effet les données d'entraînement relatives à des sports d'endurance comme la course à pied, la natation et le cyclisme. Voici les pièces d'informations principales associées à chaque séance individuelle d'entraînement:

- Date, la date et l'heure de la séance d'entraînement;
 - Type, le type de séance d'entraînement (course, nage, vélo, etc.);
 - Rythme cardiaque, le rythme cardiaque moyen enregistré au cours de la séance d'entraînement;
 - Durée, la durée de la séance d'entraînement;
 - Distance, la distance couverte pendant la séance d'entraînement (mesurée en Kilomètres ou en miles);
 - Lieu, le lieu de la séance d'entraînement;
 - Commentaires, remarques générales sur la séance d'entraînement.
- a. Sachant que toutes ces informations doivent être stockées dans un élément **séance** d'entraînement, vous déterminez lesquelles sont mieux adaptées comme éléments enfants et lesquelles seraient mieux sous la forme d'attributs?
- b. Donner ensuite le code de la DTD ETML (etml.dtd)
- c. Créer un document XML (document exemple codé en ETML) comprenant trois séances d'entraînement.

Exercice 2 :

On se propose de définir un format XML de stockage des contacts. Un contact contient le nom, le prénom, le numéro de téléphone, l'e-mail et l'adresse d'une personne. L'adresse se compose du numéro, du nom de la rue et de la ville

- a. Proposer une DTD.
- b. Créer un document XML (document exemple) comprenant deux contacts.
- c. Vérifier, à l'aide de l'éditeur que votre document est valide.

4. Série d'exercices 2 : Corrigé

Exercice 1 :

a. Conception théorique :

Voici quelques raisons logiques qui peuvent expliquer une répartition de ces informations en éléments et en attributs:

- Attributs: Date, Type, Rythme cardiaque. Il s'agit toujours de valeurs courtes et simples. L'attribut type peut être une liste énumérée de valeurs prédéfinies (course, nage, vélo, etc.).
- Éléments enfants: Durée, Distance, Lieu, Commentaires. La durée et la distance d'une séance pourraient parfaitement, en terme de modélisation, être stockées comme éléments ou comme attributs. En retenant l'option éléments on se laisse pourtant la possibilité de leur affecter des attributs supplémentaires, comme l'unité de mesure exacte.
- Le lieu et les commentaires contiennent potentiellement du texte descriptif et sont donc mieux en tant qu'éléments enfants.

b. DTD ETML (etml.dtd)

```
<!ELEMENT jouralseance (seance)+>
<!ELEMENT seance (duree, distance, lieu, commentaires)>
<!ATTLIST seance
  date CDATA #IMPLIED
  type (course | natation | cyclisme) "course"
  rythmcard CDATA #IMPLIED
>
```

```
<!ELEMENT duree (#PCDATA)>
<!ATTLIST duree
  unites (secondes | minutes | heures) "minutes"
>
```

```
<!ELEMENT distance (#PCDATA)>
<!ATTLIST distance
  unites (miles | kilometres | tours) "miles"
>
```

<!ELEMENT lieu (#PCDATA)>

<!ELEMENT commentaires (#PCDATA)>

c. Exemple de document ETML (documentETML.xml)

<?xml version="1.0"?>

<!DOCTYPE journalseance SYSTEM "etml.dtd">

<journalseance>

<seance date="11/19/05" type="course" rythmcard="158">

<duree unites="minutes">50</duree>

<distance unites="miles">5.5</distance>

<lieu>Warner Park</lieu>

<commentaires>Milieu de matinée, un peu venteux.</commentaires>

</seance>

<seance date="11/21/05" type="cyclisme" rythmcard="153">

<duree unites="heures">1.5</duree>

<distance unites="miles">26.4</distance>

<lieu>Natchez Trace Parkway</lieu>

<commentaires>Parcours en colline. Je me sens fort comme un
Bœuf.</commentaires>

</seance>

<seance date="11/24/05" type="course" rythmcard="156">

<duree unites="heures">2.5</duree>

<distance unites="miles">16.8</distance>

<lieu>Warner Park</lieu>

<commentaires>Après midi. Sensation de force honnête.</commentaires>

</seance>

</journalseance>

Exercice 2 :

a. DTD (annuaire.dtd)

<?xml version="1.0" encoding="UTF-8"?>

<!ELEMENT annuaire (contact*)>

```

<!ELEMENT contact (nom, prenom+, tel+,email*,adresse) >
<!ELEMENT nom (#PCDATA) >
<!ELEMENT prenom (#PCDATA) >
<!ELEMENT tel (#PCDATA) >
<!ELEMENT email (#PCDATA) >
<!ELEMENT adresse (numero, rue,ville) >
<!ELEMENT numero (#PCDATA) >
<!ELEMENT rue (#PCDATA) >
<!ELEMENT ville (#PCDATA) >

```

b. Document XML contenant des contacts

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE annuaire SYSTEM "annuaire.dtd">
<annuaire>
  <contact>
    <nom>AZER</nom>
    <prenom>Azer</prenom>
    <tel>0678905044</tel>
    <email>aa@gmail.com</email>
    <email>aa@yahoo.fr</email>
    <adresse>
      <numero>2</numero>
      <rue>Ouahrane</rue>
      <ville>Fes</ville>
    </adresse>
  </contact>

  <contact>
    <nom>AAIM</nom>
    <prenom>Moumen</prenom>
    <tel>0678900033</tel>
    <adresse>
      <numero>2</numero>
      <rue>Tanger</rue>
      <ville>Rabat</ville>
    </adresse>
  </contact>
</annuaire>

```

5. Série d'exercices 3 : XSD (XML Schema Language)

Exercice 1 :

On se propose de concevoir un schéma XML (XSD) pour un langage de balisage d'entraînement sportif. Ce langage de balisage qu'on va appeler ETML (Endurance Training Markup Language), pourrait être pratique si on envisage de participer à des compétitions de marathon ou de triathlon. Il modèle en effet les données d'entraînement relatives à des sports d'endurance comme la course à pied, la natation et le cyclisme. Voici les pièces d'informations principales associées à chaque séance individuelle d'entraînement:

- Date, la date et l'heure de la séance d'entraînement;
- Type, le type de séance d'entraînement (course, nage, vélo, etc.);
- Rythme cardiaque, le rythme cardiaque moyen enregistré au cours de la séance d'entraînement;
- Durée, la durée de la séance d'entraînement;
- Distance, la distance couverte pendant la séance d'entraînement (mesurée en Kilomètres ou en miles);
- Lieu, le lieu de la séance d'entraînement;
- Commentaires, remarques générales sur la séance d'entraînement.

- a. Proposer un code pour le XSD ETML (etml.xsd)
- b. Créer un document XML (document exemple codé en ETML) comprenant trois séances d'entraînement.

Exercice 2 :

Un annuaire regroupe un ensemble d'informations sur des personnes. Chaque personne est qualifiée par un code et est définie par :

- Un nom qui est décomposé en nom de famille et prénom
- Un type qui peut être soit « ami », « client », ou « fournisseur »
- Une date de naissance
- De 1 à 3 adresses email qui doivent respecter la forme d'une adresse email.
- Plusieurs numéros de téléphone. Chaque numéro est écrit sous forme d'une **liste de nombres de type byte**. Par exemple : 33 55 44 4.

- a. Proposer un schéma XML.
- b. Ecrire un document XML comprenant les informations de trois personnes.

Exercice 3 :

Soit un document XML contenant un nombre indéterminé d'éléments sous la forme :

```
<numeros>
  <contact titre="..." techno="...">
    <nom>...</nom>
    <prenom>...</prenom>
    <telephone> ...</telephone>
    <email>...</email>
    <email>...</email>
    ...
  </contact>
</numeros>
```

L'élément *telephone* et l'attribut *techno* sont en option. Les textes seront des chaînes simples **xs:string**.

- l'attribut « techno » ne pourra prendre qu'une valeur parmi : XML, Java, Autre.
- le numéro de téléphone est écrit sous forme d'une liste de 5 entiers.
- La valeur de l'email doit respecter le pattern (modèle ou format standard) d'une adresse e-mail.

- a. Créer un schéma « contact.xsd » en définissant et utilisant les types simples suivants :
 - technoType : énumération dont les valeurs possibles sont XML, Java, Autre.
 - telType : permet de restreindre le numéro de téléphone à une liste de 5 **nombre de type entier**. Créez d'abord un type nommé « **listIntType** » pour la liste d'entiers.
 - emailType : pattern `[a-z]+@[a-z]+\.[a-z]{2,3}`

6. Série d'exercices 3 : Corrigé

Exercice 1 :

a. XSD (etml.xsd)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <!-- Le type d'un élément peut être global et donc associé à un nom ou bien
  être local à
  cet élément : les lignes ci-dessous montrent un exemple
  -->

  <xsd:element name="journalseance">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="seance" type="seanceType" minOccurs="0"
          maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

  <!-- Voici une autre solution utilisant un type déclaré global -->
  <!--
  <xsd:element name="journalseance" type="journalseanceType" />

  <xsd:complexType name="journalseanceType">
    <xsd:sequence>
      <xsd:element name="seance" type="seanceType" minOccurs="0"
maxOccurs="unbounded"/>
    </xsd:sequence>
  </xsd:complexType>
  -->

  <xsd:complexType name="seanceType">
    <xsd:sequence>
      <xsd:element name="duree" type="xsd:duration"/>
      <xsd:element name="distance" type="distanceType"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:schema>
```



```

    <xsd:element name="lieu" type="xsd:string"/>
    <xsd:element name="commentaires" type="xsd:string" />
</xsd:sequence>

<xsd:attribute name="date" type="xsd:date" use="required"/>
<xsd:attribute name="type" type="typeType" use="required"/>
<xsd:attribute name="rythmcard" type="xsd:positiveInteger"/>
</xsd:complexType>

<!-- Les auteurs de XSD ont considéré que le passage d'un contenu simple
à un type complexe pouvait se faire par dérivation (extension)-->
<xsd:complexType name="distanceType">
  <xsd:simpleContent>
    <xsd:extension base="xsd:decimal">
      <xsd:attribute name="unites" type="unitesType"
        use="required"/>
    </xsd:extension>
  </xsd:simpleContent>
</xsd:complexType>

<xsd:simpleType name="unitesType">
  <xsd:restriction base="xsd:string">
    <xsd:enumeration value="miles"/>
    <xsd:enumeration value="kilometres"/>
    <xsd:enumeration value="tours"/>
  </xsd:restriction>
</xsd:simpleType>

<xsd:simpleType name="typeType">
  <xsd:restriction base="xsd:string">
    <xsd:enumeration value="course"/>
    <xsd:enumeration value="natation"/>
    <xsd:enumeration value="cyclisme"/>
  </xsd:restriction>
</xsd:simpleType>

</xsd:schema>

```

b. Exemple de document ETML (documentETML.xml)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<journalseance
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="etml.xsd">

  <seance date="2005-11-19" type="course" rythmcard="158">
    <duree>PT45M</duree>
    <distance unites="kilometres">5.5</distance>
    <lieu>Route Sefrou</lieu>
    <commentaires>Milieu de matinée, un peu venteux.</commentaires>
  </seance>

  <seance date="2005-11-21" type="cyclisme" rythmcard="153">
    <duree>PT2H30M</duree>
    <distance unites="miles">37.0</distance>
    <lieu>Forêt Ain chkaf</lieu>
    <commentaires>Parcours en colline. Je me sens fort comme un
boeuf.</commentaires>
  </seance>

  <seance date="2005-11-24" type="course" rythmcard="156">
    <duree>PT1H30M</duree>
    <distance unites="miles">8.5</distance>
    <lieu>Forêt Traat</lieu>
    <commentaires>Après midi. Sensation de force honnête.</commentaires>
  </seance>
</journalseance>
```

Exercise 2 :

a. XSD (annuaire.xsd)

```
<?xml version="1.0" encoding="UTF-8"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <xsd:element name="anuaire">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="personne" type="personneType"
minOccurs="0" maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

  <xsd:complexType name="personneType">
    <xsd:sequence>
      <xsd:element name="nom">
        <xsd:complexType>
          <xsd:attribute name="nomFamille" type="xsd:string"/>
          <xsd:attribute name="prenom" type="xsd:string"/>
        </xsd:complexType>
      </xsd:element>

      <xsd:element name="email" type="emailType" minOccurs="1"
maxOccurs="3"/>
      <xsd:element name="tel" type="telType" maxOccurs="unbounded"/>

    </xsd:sequence>

    <xsd:attribute name="code" type="xsd:int"/>
    <xsd:attribute name="dateNaissance" type="xsd:date"/>
    <xsd:attribute name="type">
      <xsd:simpleType>
        <xsd:restriction base="xsd:string">
          <xsd:enumeration value="ami"/>
          <xsd:enumeration value="client"/>
          <xsd:enumeration value="fournisseur"/>
        </xsd:restriction>
      </xsd:simpleType>
    </xsd:attribute>
  </xsd:complexType>
</xsd:schema>
```

```

    </xsd:attribute>
</xsd:complexType>

<xsd:simpleType name="emailType">
  <xsd:restriction base="xsd:string">
    <xsd:pattern value="[a-z]+@[a-z]+\.[a-z]{2,3}" />
  </xsd:restriction>
</xsd:simpleType>

<xsd:simpleType name="telType">
  <xsd:list itemType="xsd:unsignedByte">
  </xsd:list>
</xsd:simpleType>
</xsd:schema>

```

b. Exemple de document XML (annuaire.xml)

```

<?xml version="1.0" encoding="UTF-8"?>
<anuaire xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="annuaire.xsd">

  <personne code="1" dateNaissance="2008-02-29" type="client">
    <nom nomFamille="Dadi" prenom="Hassan"/>
    <email>dzt@aze.com</email>
    <tel>33 55 44 4</tel>
  </personne>

  <personne code="2" dateNaissance="2008-02-29" type="ami">
    <nom nomFamille="Rannabi" prenom="Latif"/>
    <email>dzt@aze.com</email>
    <tel>33 55 44 4</tel>
  </personne>

</anuaire>

```

Exercice 3 :

a. XSD (contact.xsd)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xs:schema
  xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="numeros" type="numerosType"/>

  <xs:complexType name="numerosType">
    <xs:sequence>
      <xs:element name="contact" type="contactType"
maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="contactType">
    <xs:sequence>
      <xs:element name="nom" type="xs:string"/>
      <xs:element name="prenom" type="xs:string"/>
      <xs:element name="telephone" type="telType" minOccurs="0"/>
      <xs:element name="email" type="emailType"
maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="titre" type="xs:string" use="required"/>
    <xs:attribute name="techno" type="technoType" use="optional"/>
  </xs:complexType>

  <xs:simpleType name="technoType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="XML"/>
      <xs:enumeration value="Java"/>
      <xs:enumeration value="Autre"/>
    </xs:restriction>
  </xs:simpleType>

  <xs:simpleType name="listIntType">
    <xs:list itemType="xs:int">
    </xs:list>
  </xs:simpleType>
```

```
<xs:simpleType name="telType">
  <xs:restriction base="listIntType">
    <xs:length value="5"/>
  </xs:restriction>
</xs:simpleType>

<xs:simpleType name="emailType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[a-z]+@[a-z]+\.[a-z]{2,3}"/>
  </xs:restriction>
</xs:simpleType>

</xs:schema>
```

Références

- Modélisation XML, Lonjon, A. and Thomasson, J.J. and Maesano, L., isbn=9782212115215}, year=2006, Editions Eyrolles
- XML pour l'édition, Bernard Prost, Jul 2011, Editions Eyrolles
- XML: cours et exercices. Modélisation, Schémas et DTD, design patterns, XSLT, DOM, Relax NG, XPath, SOAP, XQuery, XSL-FO, SVG, eXist, Brillant, A., isbn= 9782212121513, 2007, Editions Eyrolles